



✓ Resumo Executivo da Parceria

Este documento apresenta a especificação técnica para que a **LiveTip** integre o catálogo de efeitos e memes sonoros da **PlonkMemes** em sua plataforma de doações (Pix, Tips e Alertas sonoros) para streamers no **OBS Studio** e **Streamlabs**.

1 Diretrizes e Compliance de Streaming (Anti-DMCA)

1.1. Licença Comercial para Streamers

Todos os áudios disponibilizados pela API v1 e identificados como seguros estão autorizados para execução pública em transmissões ao vivo monetizadas e gravação permanente de VODs na **LiveTip** e plataformas parceiras (Twitch, YouTube Gaming, Kick, TikTok Live).

1.2. Proteção Anti-Content ID (YouTube / Twitch Safe)

Para proteger criadores contra silenciamento de áudio em VODs ou perda de monetização, a API dispõe de curadoria algorítmica de fonogramas protegidos:

- A LiveTip deve sempre realizar consultas com o parâmetro: `GET /api/v1/sounds?safe_only=true`.
- Músicas comerciais registradas em gravadoras e fonogramas sob direitos autorais estritos são filtrados automaticamente.

1.3. Espelhamento e Cache Soberano (S3 / Cloudflare R2)

A **LiveTip** possui **permissão explícita** para realizar cache, espelhamento e download dos arquivos `.mp3` diretamente para seu próprio storage de CDN caso opte por controle local de tráfego e latência.

2 Requisitos Técnicos de Áudio & OBS (CEF)

⚠ Particularidade Crítica do OBS Studio (CEF)

O navegador interno do OBS Studio (Chromium Embedded Framework) requer suporte a cabeçalhos `Range: bytes` (HTTP 206) e CORS irrestrito para calcular o buffer sem congelar a reprodução do alerta.

- Formato do Arquivo:** MP3 (`audio/mpeg`), 44.1 kHz, 16-bit, bitrate médio 128 kbps a 192 kbps.
- CORS:** Cabeçalho `Access-Control-Allow-Origin: *` habilitado globalmente na API e na CDN.
- HTTP 206 Partial Content:** Suporte comprovado a Range Requests para pré-carregamento imediato.
- Normalização de Volume (EBU R128):** Teto recomendado de **-16 LUFS** no player de overlay para evitar que memes com pico elevado prejudiquem o áudio da transmissão.

3 Documentação da API REST (v1)

Base URL de Produção: `https://plonkmemes.lol/api/v1` (Fallback: `https://plonkmemes.vercel.app/api/v1`)

3.1. Listar Catálogo de Sons (`GET /api/v1/sounds`)

Parâmetro	Tipo	Padrão	Descrição
<code>safe_only</code>	boolean	<code>false</code>	Obrigatório para LiveTip: Retorna exclusivamente sons livres de Content ID.
<code>q</code>	string	-	Busca textual por título ou tags (ex: <code>pou</code> , <code>grosso</code>).
<code>category</code>	string	<code>todos</code>	Filtra por categoria (<code>memes</code> , <code>efeitos</code> , <code>jogos</code> , etc.).
<code>trending</code>	boolean	<code>false</code>	Filtra apenas sons em alta.
<code>page</code> / <code>limit</code>	integer	1 / 50	Paginação de resultados (limite máx: 100 por requisição).

Exemplo de Resposta (200 OK):

```
{
  "data": [
    {
      "id": "pou-estourado-48183",
      "title": "POU ESTOURADO",
      "slug": "pou-estourado-48183",
      "audio_url": "https://bfwdlanqfokvmxhzfdie.supabase.co/storage/v1/object/public/sounds/pou-estourado-481",
      "category": "efeitos",
      "duration_seconds": 2.0,
      "is_trending": true,
      "is_safe_for_streamers": true,
      "tags": ["pou", "estourado"]
    }
  ],
  "pagination": { "page": 1, "limit": 1, "total": 2266, "total_pages": 2266 }
}
```

3.2. Detalhes de um Som ([GET /api/v1/sounds/:id](#))

Retorna metadados e URL de áudio direta através do ID ou slug do som.

3.3. Sincronização de Depreciações ([GET /api/v1/sounds/deprecations](#))

Parâmetro `?since=ISO_TIMESTAMP` . Permite à LiveTip inativar periodicamente sons descontinuados ou removidos.

4 Snippet: Player de Overlay para OBS Studio (Zero Latency)

Implementação leve com limitador de volume dinâmico (DynamicsCompressor) para proteger os ouvidos do streamer:

```
// Browser Source OBS - Web Audio com proteção de pico auditivo
const audioCtx = new (window.AudioContext || window.webkitAudioContext)();

const compressor = audioCtx.createDynamicsCompressor();
compressor.threshold.setValueAtTime(-18, audioCtx.currentTime);
compressor.ratio.setValueAtTime(12, audioCtx.currentTime);

const masterGain = audioCtx.createGain();
masterGain.gain.setValueAtTime(0.85, audioCtx.currentTime); // Volume padrão 85%

compressor.connect(masterGain);
masterGain.connect(audioCtx.destination);

async function triggerLiveTipAlert(audioUrl) {
  if (audioCtx.state === 'suspended') await audioCtx.resume();

  const response = await fetch(audioUrl, { mode: 'cors' });
  const arrayBuffer = await response.arrayBuffer();
  const audioBuffer = await audioCtx.decodeAudioData(arrayBuffer);

  const source = audioCtx.createBufferSource();
  source.buffer = audioBuffer;
  source.connect(compressor);
  source.start(0);
}
```